

Managing a pharmacy by hand is something that worked a long time ago, but today it causes more problems than it solves. Most small pharmacies in Pakistan still rely on handwritten records or simple spreadsheets to keep track of their medicines, sales, and stock. This leads to a number of serious issues that affect both the pharmacy staff and the customers who depend on them.

The most common problem is that medicines expire on the shelves without anyone noticing. When stock is recorded manually, it is easy for a batch of medicines to sit at the back of the shelf while newer stock gets sold. Nobody checks the expiry dates regularly, and by the time someone does, the expired medicines have already caused losses. In some cases they are accidentally sold to patients, which is a serious health risk.

Another issue is running out of stock without warning. A pharmacist might sell the last few units of a medicine and not realize it until a customer comes in asking for it the next day. There is no system in place to flag when a product is getting low before it actually runs out. This affects sales, customer trust, and the overall reputation of the pharmacy.

When a customer asks for a specific medicine that is not available, the staff currently has no way to quickly suggest an alternative from the same drug family. The pharmacist either has to rely on memory or send the customer away, both of which are unprofessional and inconvenient.

On the management side, there is no proper separation between what a manager can do and what a regular employee can do. Any staff member can change prices or delete records, which creates opportunities for errors or misuse. There is also no log of who did what, so if something goes wrong there is no way to trace it back.

Sales records are either written in notebooks or kept in basic spreadsheets that are not connected to the actual stock. This means a sale can be recorded without the stock being reduced, leading to inaccurate inventory counts. Generating any kind of sales report at the end

of the day or month requires going through everything manually, which takes hours.

The problem, in short, is that there is no organized system connecting all these parts together. The pharmacy runs on disconnected pieces of information, and that is exactly what this project aims to fix.

Chapter 2

Goals

The main goal of this project is to build a working pharmacy management system in Java that brings all the essential operations of a pharmacy under one program. The system is not meant to be complicated. It is meant to be practical, organized, and something a real pharmacy could actually use.

Inventory Management

The system should store complete information about each medicine, including its name, the company that supplies it, its price, its expiry date, the current quantity in stock, and the category of drug formula it belongs to. All this information should be saved in a file so that nothing is lost when the program is closed.

Connected Sales System

When a sale is made, the stock should reduce automatically. A bill should be generated for every transaction, showing the medicine name, quantity, price, and total amount. If a discount is applied, it should appear on the bill. Every sale should be saved to a log file so there is always a complete record.

Role-Based Access Control

There should be three types of users. The admin has full access, including adding and deleting medicines, changing prices, and viewing reports. The employee can search, sell, and view inventory but cannot make changes to prices or delete records. The customer can browse available medicines and check prices without needing to log in at all.

Automatic Alert System

Every time the program opens, it should check which medicines are expiring within the next thirty days and which medicines have fallen below their minimum stock level. All warnings should be shown before the main menu appears so that nothing gets missed.

Alternative Medicine Suggestions

If a customer searches for a medicine that is out of stock, the system should look through the inventory for other medicines with the same drug formula and suggest them. This way the customer is not simply told the medicine is unavailable but is offered something practical they can use instead.

File-Based Data Persistence

All data should be saved in files at all times. The inventory file should update whenever a medicine is added, changed, sold, or deleted. A separate file should keep a record of every transaction. An activity log should record every admin action. A dated backup of the inventory should be created automatically every time the program closes, so previous states of the inventory can always be recovered.

Chapter 3

Proposed Idea

The idea behind this project came from observing how small pharmacies in our surroundings actually work. Most of them have no digital system at all. When a customer comes in asking for a medicine, the pharmacist either remembers where it is or goes looking for it. When stock runs out, they find out because a customer asks for something and the shelf is empty. When medicines expire, they sometimes find out too late.

We thought about what kind of program would actually help with this, keeping in mind that it needs to be simple enough for a regular pharmacy to use without any technical training. The idea we came up with is a console-based Java application that can be run on any computer

without any additional installation or software.

The program starts by asking whether the user is an admin, an employee, or a customer. From there, each type of user is given a menu suited to what they are allowed to do. The customer menu is the simplest, covering only search and browse. The employee menu adds the ability to sell medicines and generate bills. The admin menu gives full control over the entire system.

One idea contributed by a team member was the alternative medicine suggestion feature. Instead of simply showing that a medicine is out of stock, the system checks whether any other medicine in the inventory shares the same drug formula and displays those options to the customer. For example, if Panadol is unavailable, the system would find Calpol or Paracet and suggest them, since all three contain Paracetamol as the active ingredient. This is a small feature but it makes the system genuinely useful in practice.

The inventory is stored in a plain text file with each medicine on a separate line. This approach does not require any database software and keeps the system lightweight and portable. The same method is used for the sales log and the activity log.

The alert system came from thinking about what a pharmacist actually needs to know before starting the day. Before the main menu appears, the program checks the inventory and highlights anything that needs immediate attention. This way critical information is never buried inside a menu that someone might forget to check.

The concept of separating admin access from employee access reflects how a real pharmacy operates. The owner controls pricing and records. The staff handles sales. The activity log creates accountability. These are not just technical decisions — they are decisions based on how the business actually works.

The idea as a whole is to build something that solves real problems using the concepts learned in this course, including inheritance, encapsulation, polymorphism, abstraction, and file handling — not as isolated exercises but as parts of one working system.

The proposed solution is a Java-based console application built around eight main classes, each responsible for a specific part of the system. The design follows the object-oriented principles covered in this course, with a clear separation between classes so that each part can be built, tested, and explained independently.

Medicine.java

This class is the foundation of the entire system. It stores all details of a medicine including name, company, price, expiry date, current quantity, minimum stock threshold, drug formula category, and a flag for prescription requirement. It provides methods to add, update, delete, and search medicines. The full inventory is stored as an array of Medicine objects. This class also handles the alternative medicine suggestion by scanning the array for medicines that share the same formula category when a requested item is out of stock.

FileManager.java

This class handles all reading and writing of files. No other class in the system accesses files directly. FileManager loads the inventory when the program starts, saves it when the program ends, logs every sale transaction, records every admin action with a timestamp, stores alert history, and creates a dated backup copy on shutdown. Keeping all file operations in a single class ensures consistency and makes errors easy to trace.

Sales.java and Bill.java

Sales.java manages the process of selling a medicine. It checks whether sufficient stock is available, creates a Bill object for the transaction, reduces the quantity in the Medicine inventory, and instructs FileManager to log the sale. Bill.java stores the details of a single transaction — medicine name, quantity sold, price per unit, discount applied, total amount, date of sale, and the name of the staff member who processed it. It also provides a method to print a formatted receipt on screen.

User.java, Admin.java, Staff.java, Customer.java

User is an abstract class that defines the structure shared by all user types, with fields for username, password, and role, and an abstract login method that each subclass must implement. Admin extends User with full access — price changes, deletions, activity log viewing, and discount application. Staff extends User with restricted access limited to searching, viewing, and selling. Customer requires no login and can search the inventory, view medicines, and receive alternative suggestions when a product is unavailable.

AlertSystem.java

This class runs automatically every time the program starts. It checks the full inventory for medicines expiring within thirty days and for items whose stock has dropped below the set minimum. All alerts are displayed before the main menu appears and are also saved to an alert history file through FileManager.

Menu.java

This is the entry point of the program and the only class that handles user input and screen output. It uses a Scanner to read input and displays the appropriate menu based on the logged-in user. On startup it calls FileManager to load the inventory and AlertSystem to run checks. On shutdown it saves the inventory and creates a backup. All user choices are routed from Menu to the appropriate class and method. This design means that when a graphical interface is added in the future, only this class needs to be replaced — all backend classes remain unchanged.

Data Flow Overview

When a customer searches for a medicine, Menu calls Medicine to perform the search. If the item is out of stock, Medicine calls its own alternative suggestion method and returns the results to Menu for display. When an employee sells a medicine, Menu calls Sales, which checks stock through Medicine, creates a Bill, reduces stock, and tells FileManager to log the sale. When the admin changes a price, Menu verifies the user role, calls Admin to update Medicine, and FileManager saves and logs the action. Every part of the system does exactly one job, all classes connect in a logical sequence, and the code stays easy to understand, test, and extend.

The project will be completed over four weeks, divided into phases. Each phase has a clear goal so that the team can track progress and make sure no part of the work is left to the last minute. The workload is distributed across four working members, each responsible for specific classes, so that all four can work in parallel without depending on each other except where one class requires another to be completed first.

Phase 1 — Core Backend (Week 1)

The first week focuses on the two most important classes that everything else depends on. `Medicine.java` will be completed with all fields, methods, and the alternative medicine suggestion logic. `FileManager.java` will be completed with all read and write operations. The abstract `User` class along with `Admin`, `Staff`, and `Customer` will also be built during this week as they are independent and can run in parallel. The project folder structure will be set up and shared with the full team.

Phase 2 — Sales and Alerts (Week 2)

The second week covers the transactional side of the system. `Sales.java` and `Bill.java` will be built and tested using the `Medicine` and `FileManager` classes from week one. `AlertSystem.java` and `DateHelper.java` will also be completed during this phase. By the end of week two, all individual backend classes should be working and tested on their own.

Phase 3 — Integration and Console Testing (Week 3)

The third week focuses on connecting all classes through `Menu.java`. The full startup sequence, shutdown sequence, login system, and all three user menus will be implemented. The entire team will test the console version together, going through every possible scenario including adding medicines, selling, searching, triggering alerts, and using the alternative medicine feature. Any issues found will be fixed before the final phase.

Phase 4 — GUI and Submission (Week 4)

The fourth week focuses on adding a graphical user interface using Java Swing once the topic is covered in class. Since Menu.java was designed as a separate layer, the GUI can be added by replacing the Scanner-based interface with Swing windows without changing any backend class. The final report will be completed with screenshots of the working system. The complete project including code, report, and proposal will be submitted by the deadline.

Chapter 6

Conclusion

The Pharmacy Management System is a project that came from a real problem. Small pharmacies in Pakistan lose money to expired stock, miss sales because of untracked inventory, and operate with no record-keeping system that can be referred to later. The goal of this project is to address all of these problems with a single, well-organized Java application.

The system handles three types of users with different levels of access, which reflects how a real pharmacy actually works. The admin controls everything. The employees handle day-to-day sales. The customers browse and buy without needing an account. This separation makes the system both secure and straightforward to use.

The alternative medicine feature makes the system more useful than a basic stock tracker because it actively helps the customer find what they need, even when the exact product is unavailable. This kind of practical detail is what makes a system worth using in the real world rather than just in a lab.

From a technical standpoint, the project covers the major concepts from this course. Inheritance and polymorphism appear in the user hierarchy. Abstraction is applied through the base User class. Encapsulation is used throughout by keeping all fields private with controlled access through methods. File handling provides data persistence. Exception handling will be applied to all file operations and user inputs.

The project is divided between team members in a way that is both fair and purposeful. Each working member owns specific classes, understands them fully, and can explain them to the teacher independently. The member handling documentation contributes equally by writing the proposal, preparing the report, and creating the presentation — work that is just as important as the code itself.

By the end of this project, the team expects to have a fully working pharmacy management system that could realistically be used by a small pharmacy, a solid understanding of how object-oriented design functions in practice, and the experience of building something complete as a team where every part connects logically to every other part.

This is not just an assignment. It is a chance to build something that solves a real problem and to learn from actually doing it.